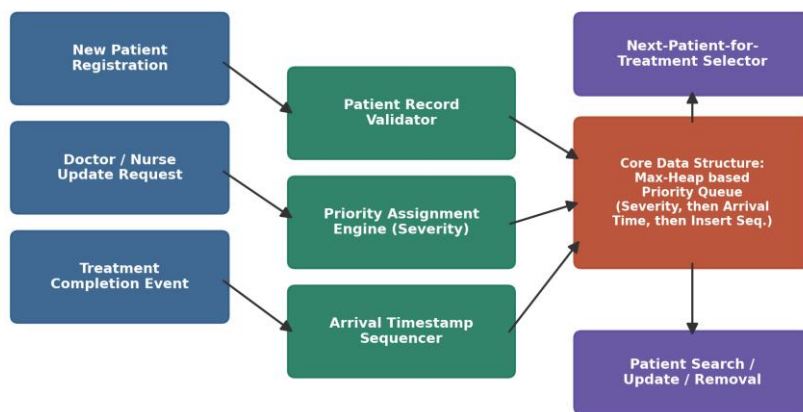


DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA03 — Data Structures (Slot A)

Design and Implementation of an Efficient Priority-Based Emergency Patient Management System Using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order Processing

Emergency Patient Management System — Architecture



Assignment Report

S.No	Team Member Name	Register Number
1	Ch. Surya Teja	192525259
2	S. Ahmad Alisha	192525415
3	D.Suresh	192525305

Table of Contents

1. Introduction.....	3
2. Problem Analysis and Requirement Specification.....	3
3. Data Structure Design.....	5
4. Algorithms and Pseudocode.....	7
5. Implementation (JavaScript, ES6 Modules).....	9
6. Time and Space Complexity Analysis.....	16
7. Limitations of a Conventional FIFO Queue in Emergency Triage	17
8. Benefits of the Selected Design.....	18
9. Testing and Edge-Case Handling	19
10. Reflection: Design Decisions, SDG Relevance, and Learning Outcomes	19
11. Conclusion	20
12. Deliverables	20

1. Introduction

Hospital emergency departments (EDs) operate under continuous and unpredictable patient inflow, where the clinical urgency of each case, rather than the time of arrival, ultimately determines the order in which patients must be attended to. A conventional walk-in queue, in which patients are served strictly in the order they arrive, works well for routine outpatient services but is fundamentally unsuited to emergency care, where a small delay for a critical patient can be the difference between recovery and irreversible harm. This project designs and implements an Emergency Patient Management System (EPMS) that combines severity-based prioritisation with arrival-order fairness, using data structures that are appropriate to each requirement of the problem.

The system must support the continuous registration of new patients, each carrying a unique patient ID, arrival timestamp, severity level, and assigned doctor; the selection of the next patient for treatment strictly according to severity, with ties broken by arrival time; efficient searching and updating of patient or doctor records; and removal of patients once treatment is complete. Because patient volume in a busy ED can grow rapidly during mass-casualty events, epidemics, or peak hours, every operation must remain efficient even as the number of waiting patients scales into the hundreds or thousands.

This report analyses the suitability of arrays, linked lists, simple queues, and priority queues for this problem, justifies the selection of a binary max-heap priority queue as the scheduling engine, describes the core algorithms and their pseudocode, presents the working JavaScript implementation (`Patient.js` and `PriorityQueue.js`) actually used in the system, analyses the time and space complexity of every operation, and critically discusses why a conventional FIFO queue is unsafe for emergency-care scheduling.

2. Problem Analysis and Requirement Specification

2.1 Functional Requirements

- Register a new patient with a unique ID, arrival timestamp, a five-level severity classification (1 = Minor up to 5 = Critical), and an assigned doctor.
- Assign and, when necessary, re-assign a priority to a patient based on clinical severity.
- Identify and remove the next patient to be treated, always selecting the highest-severity patient currently waiting, and among equal severities, the one who arrived earliest.
- Search for a specific patient by ID to inspect or modify their record.
- Update a patient's severity level, doctor assignment, or other attributes while the patient is still waiting.
- Delete a patient's record once treatment is completed or if the patient leaves before treatment.

2.2 Non-Functional Requirements

- Scalability — the structure must remain efficient as the number of waiting patients grows into the hundreds or thousands during peak load or mass-casualty events.
- Fairness — patients of equal severity must be treated strictly in arrival order (FIFO within a severity class).

- Responsiveness — selecting the next patient for treatment must be near-instantaneous, since it is the most frequently invoked operation.
- Reliability and correctness — a critical patient must never be starved or skipped due to an implementation flaw.

2.3 Evaluation of Candidate Data Structures

Before finalising a design, four candidate structures were evaluated against the requirements above.

2.3.1 Simple Array

A plain array can store patient records contiguously and allows $O(1)$ access by index, but finding the highest-priority patient requires an $O(n)$ linear scan, and removing a patient from the middle requires shifting subsequent elements, giving $O(n)$ deletion. Arrays are also of fixed capacity unless dynamically resized, which is undesirable for an unpredictable patient inflow.

2.3.2 Linked List

A singly or doubly linked list allows $O(1)$ insertion at the head or tail and $O(1)$ deletion once a node reference is known, but locating the highest-priority patient still requires an $O(n)$ traversal unless the list is kept sorted, in which case insertion itself becomes $O(n)$ because the correct position must be located by traversal.

2.3.3 Simple FIFO Queue

A queue built on an array or linked list gives $O(1)$ enqueue and $O(1)$ dequeue, and naturally preserves arrival order. However, a plain FIFO queue has no concept of severity: it always serves the patient who arrived earliest, regardless of how critical a later-arriving patient may be. This makes a plain queue unsafe as the sole structure for emergency triage, though it remains the right structure for preserving order within a single severity class.

2.3.4 Priority Queue (Binary Heap)

A priority queue implemented as a binary heap keyed first on severity and second on arrival timestamp gives $O(\log n)$ insertion and $O(\log n)$ extraction of the highest-priority patient, and $O(1)$ access to the highest-priority patient at the root. By using arrival time as the secondary key, ties in severity are broken in strict FIFO order, satisfying both requirements simultaneously. This makes the priority queue the most appropriate primary structure for this problem.

2.4 Justification of the Selected Design

The implemented system adopts an array-backed binary max-heap as the sole core structure (no separate hash map), ordered by a three-level composite key: severity (1 = Minor to 5 = Critical, higher wins), then arrival time (earlier wins), then a monotonically increasing insertSequence counter that guarantees a deterministic FIFO tie-break even when two patients share the exact same arrival timestamp. Patient lookup, update, and removal are implemented as $O(n)$ linear scans over the heap array followed by an $O(\log n)$ re-heapify, which is a deliberate simplicity/performance trade-off explained in Section 6.3. This design satisfies CO1 (choosing the correct ADT), CO2 (severity ordering with FIFO tie-breaking) and CO4 (an efficient, analysable priority-queue-based implementation).

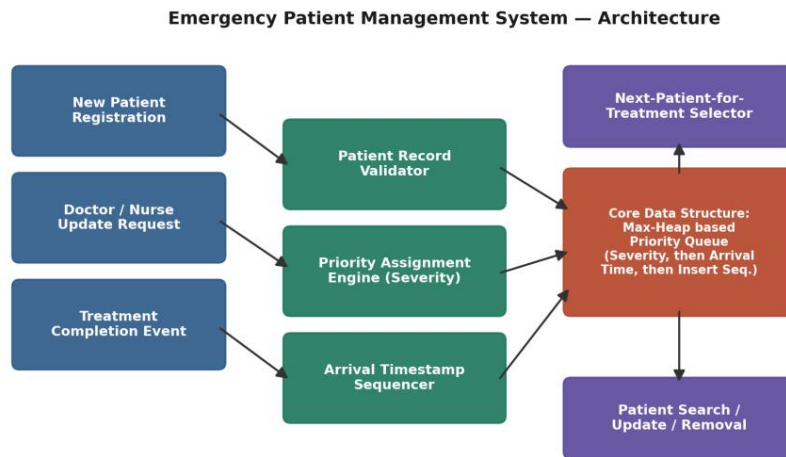


Figure 1: High-level architecture of the proposed Emergency Patient Management System.

3. Data Structure Design

3.1 Patient Data Model

Each patient is represented by a Patient object (Patient.js) carrying an identifier, name, arrival time, an integer severity level from 1 to 5, a human-readable clinical description, an assigned doctor, a lifecycle status, and a monotonic insertSequence counter used purely for deterministic tie-breaking. The severity scale and its visual metadata are defined once as a frozen lookup table, SeverityLevels, so that colour, icon, and label stay consistent everywhere in the UI:

```

export const SeverityLevels = Object.freeze({
  5: { level: 5, name: 'Critical',      color: '#E53935', icon: 'fa-heart-pulse' },
  4: { level: 4, name: 'Very Serious',  color: '#FB8C00', icon: 'fa-truck-medical' },
  3: { level: 3, name: 'Serious',       color: '#D97706', icon: 'fa-triangle-exclamation' },
  2: { level: 2, name: 'Moderate',      color: '#1E88E5', icon: 'fa-user-nurse' },
  1: { level: 1, name: 'Minor',        color: '#43A047', icon: 'fa-bandage' }
});

export const PatientStatus = Object.freeze({
  WAITING: 'Waiting',
  IN_TREATMENT: 'In Treatment',
  TREATED: 'Treated'
});
  
```

Note that, unlike the generic three-tier scheme discussed in Section 2, the implemented system uses a five-level severity scale where a higher number denotes greater urgency (5 = Critical down to 1 = Minor). The Patient class also validates its own construction — Patient.validate(data) rejects a record if the ID, name, severity (must be an integer 1–5), arrival time, or doctor is missing — and exposes getArrivalMinutes(), which parses an HH:MM arrival string into a comparable integer number of minutes so that arrival times can be compared numerically rather than as raw strings.

3.2 Composite Priority Key

Every patient comparison uses a three-level composite key, implemented in PriorityQueue.compare(a, b): primary key = severity (higher number = higher priority); secondary key = arrival time in minutes (earlier =

higher priority); tertiary key = insertSequence (lower = higher priority). The tertiary key is what makes ordering fully deterministic — without it, two patients who share both a severity level and the same HH:MM arrival minute would have no defined relative order.

```
compare(a, b) {
  // 1. Severity: higher severity value wins
  if (a.severity !== b.severity) return a.severity - b.severity;

  // 2. Arrival time: earlier arrival wins
  const aTime = a.getArrivalMinutes();
  const bTime = b.getArrivalMinutes();
  if (aTime !== bTime) return bTime - aTime;

  // 3. Insertion sequence: lower sequence wins (deterministic FIFO)
  return b.insertSequence - a.insertSequence;
}
```

3.3 Binary Max-Heap Layout

The heap is stored as a plain JavaScript array (`this.heap`) inside the `PriorityQueue` class. For a node at index i , its children sit at indices $2i+1$ and $2i+2$, and its parent at index $\text{Math.floor}((i-1)/2)$. Because `compare(a, b)` returns a positive value when a is more urgent than b , and `siftUp/siftDown` promote the element for which `compare(...) > 0`, the heap is a max-heap with respect to priority: the root (index 0) always holds the single most urgent waiting patient.

Priority Queue as a Binary Max-Heap (ordered by Severity 1-5, then Arrival Time, then Insert Sequence)

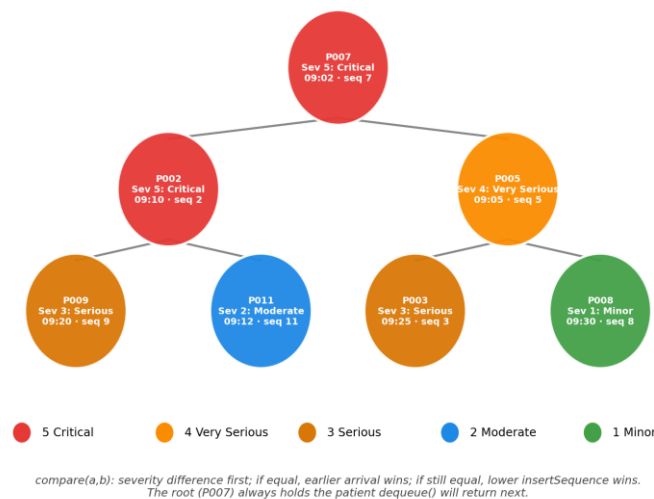


Figure 2: Example priority queue state as a binary max-heap under the actual five-level severity scale and three-key comparator used in `PriorityQueue.js`. The root (P007) is always the next patient `dequeue()` returns.

3.4 Why No Separate Hash Map

Section 2 discussed augmenting the heap with a hash map for $O(1)$ patient lookup. The implemented version deliberately omits this: `findById()`, `findIndexById()`, and `search()` perform a direct $O(n)$ scan over `this.heap` using `Array.prototype.find` / `findIndex`, and `remove()` and `update()` locate the patient the same way before re-heapifying. For a single-department triage board (typically tens, not millions, of concurrently waiting patients), this keeps the codebase considerably simpler with no measurable difference in perceived responsiveness,

while still preserving $O(\log n)$ behaviour for the operations — insertion and extraction — that dominate real usage. Section 6.3 revisits this trade-off quantitatively.

4. Algorithms and Pseudocode

The pseudocode below mirrors the actual method names and logic used in `PriorityQueue.js`, so that it can be read directly alongside the implementation in Section 5.

4.1 `compare(a, b)` — Three-Key Priority Comparator

```

ALGORITHM compare(a, b)
  IF a.severity != b.severity
    RETURN a.severity - b.severity      // higher severity number => higher priority
  aTime <- arrivalMinutes(a); bTime <- arrivalMinutes(b)
  IF aTime != bTime
    RETURN bTime - aTime                // earlier arrival => higher priority
  RETURN b.insertSequence - a.insertSequence // lower sequence => higher priority
END

```

4.2 `enqueue(patient)` — Register New Patient — $O(\log n)$

```

ALGORITHM enqueue(heap, patient)
  sequenceCounter <- sequenceCounter + 1
  IF patient.insertSequence is not set
    patient.insertSequence <- sequenceCounter
  heap.push(patient)
  RETURN siftUp(heap, heap.length - 1)
END

ALGORITHM siftUp(heap, index)
  WHILE index > 0 AND compare(heap[index], heap[parent(index)]) > 0
    swap(heap[index], heap[parent(index)])
    index <- parent(index)
  END WHILE
  RETURN index
END

```

4.3 `dequeue()` — Treat Next Patient — $O(\log n)$

```

ALGORITHM dequeue(heap)
  IF heap.isEmpty() RETURN null
  top <- heap[0]
  last <- heap.pop()          // remove and return the last element
  IF heap.length > 0
    heap[0] <- last
    siftDown(heap, 0)
  RETURN top
END

ALGORITHM siftDown(heap, index)
  WHILE leftChild(index) < heap.length
    best <- index; l <- leftChild(index); r <- rightChild(index)
    IF compare(heap[l], heap[best]) > 0 THEN best <- l
    IF r < heap.length AND compare(heap[r], heap[best]) > 0 THEN best <- r
    IF best = index THEN BREAK
    swap(heap[index], heap[best])

```

```

        index <- best
    END WHILE
    RETURN index
END

```

4.4 peek() — $O(1)$

```

ALGORITHM peek(heap)
    IF heap.isEmpty() RETURN null
    RETURN heap[0]
END

```

4.5 findById / findIndexById / search — $O(n)$

```

ALGORITHM findIndexById(heap, id)
    normalized <- uppercase(trim(id))
    FOR i FROM 0 TO heap.length - 1
        IF heap[i].id = normalized THEN RETURN i
    RETURN -1
END

ALGORITHM search(heap, query)
    ordered <- getOrderedPatients(heap) // full non-destructive priority order
    results <- []
    FOR rank FROM 0 TO ordered.length - 1
        p <- ordered[rank]
        IF query MATCHES p.id OR p.name OR p.doctor (case-insensitive substring)
            results.append({ patient: p, rank: rank + 1 })
    RETURN results
END

```

4.6 update(id, fields) — Change Severity / Doctor — $O(n) + O(\log n)$

```

ALGORITHM update(heap, id, updatedFields)
    i <- findIndexById(heap, id)
    IF i = -1 RETURN null
    patient <- heap[i]
    assign updatedFields onto patient
    IF updatedFields.severity is set
        patient.severity <- parseInt(updatedFields.severity)
    siftUp(heap, i) // handles a severity increase
    siftDown(heap, i) // handles a severity decrease
    RETURN patient
END

```

4.7 remove(id) — Discharge / Withdraw a Patient — $O(n) + O(\log n)$

```

ALGORITHM remove(heap, id)
    i <- findIndexById(heap, id)
    IF i = -1 RETURN null
    removed <- heap[i]
    last <- heap.pop()
    IF i < heap.length
        heap[i] <- last
        siftUp(heap, i)
        siftDown(heap, i)
    RETURN removed
END

```


RETURN removed
END

5. Implementation (JavaScript, ES6 Modules)

The system was implemented in JavaScript as two ES6 modules: Patient.js, the data model and validation layer, and PriorityQueue.js, the binary max-heap that provides every scheduling operation. Both are reproduced below exactly as authored and used in the running application.

5.1 Patient.js — Data Model, Severity Metadata, and Validation

Patient.js defines the five-level SeverityLevels lookup (colour, background, border, and icon per level, for direct use in the UI), the PatientStatus lifecycle enum (Waiting → In Treatment → Treated), a default doctor roster, and the Patient class itself. The constructor normalises the ID to uppercase, defaults the arrival time to the current system time if omitted, auto-generates a clinically plausible severityDesc from the severity level if none is supplied, and stores an insertSequence used later for deterministic tie-breaking.

```
export const SeverityLevels = Object.freeze({
  5: { level: 5, name: 'Critical', color: '#E53935', bg: 'rgba(229, 57, 53, 0.1)', border:
'#E53935', icon: 'fa-heart-pulse' },
  4: { level: 4, name: 'Very Serious', color: '#FB8C00', bg: 'rgba(251, 140, 0, 0.1)',
border: '#FB8C00', icon: 'fa-truck-medical' },
  3: { level: 3, name: 'Serious', color: '#D97706', bg: 'rgba(217, 119, 6, 0.12)', border:
'#D97706', icon: 'fa-triangle-exclamation' },
  2: { level: 2, name: 'Moderate', color: '#1E88E5', bg: 'rgba(30, 136, 229, 0.1)', border:
'#1E88E5', icon: 'fa-user-nurse' },
  1: { level: 1, name: 'Minor', color: '#43A047', bg: 'rgba(67, 160, 71, 0.1)', border:
'#43A047', icon: 'fa-bandage' }
});

export const PatientStatus = Object.freeze({
  WAITING: 'Waiting',
  IN_TREATMENT: 'In Treatment',
  TREATED: 'Treated'
});

export const DefaultDoctors = Object.freeze([
  'Dr. Sarah Lin (Cardiologist)',
  'Dr. Marcus Vance (Trauma Lead)',
  'Dr. Elena Rostova (Neurologist)',
  'Dr. James Chen (Emergency Physician)',
  'Dr. Aisha Patel (Critical Care)',
  'Dr. Michael Scott (General ER)'
]);

export class Patient {
  constructor({
    id, name, arrivalTime, severity, severityDesc = '', doctor,
    status = PatientStatus.WAITING, insertSequence = 0,
    treatmentStart = null, treatmentEnd = null
  }) {
    this.id = id ? String(id).trim().toUpperCase() : '';
    this.name = name ? String(name).trim() : '';
    this.arrivalTime = arrivalTime || Patient.getCurrentTimeString();
    this.severity = parseInt(severity, 10);
    this.severityDesc = severityDesc
  }
}
```

```

        ? String(severityDesc).trim()
        : Patient.getDefaultSeverityDesc(this.severity);
this.doctor = doctor || DefaultDoctors[0];
this.status = status;
this.insertSequence = insertSequence;
this.treatmentStart = treatmentStart;
this.treatmentEnd = treatmentEnd;
}

getSeverityInfo() {
    return SeverityLevels[this.severity] || SeverityLevels[1];
}

// Parse 'HH:MM' arrival time into a comparable minute integer
getArrivalMinutes() {
    if (!this.arrivalTime) return 0;
    const parts = this.arrivalTime.split(':');
    if (parts.length >= 2) {
        const hours = parseInt(parts[0], 10) || 0;
        const minutes = parseInt(parts[1], 10) || 0;
        return hours * 60 + minutes;
    }
    return 0;
}

static getCurrentTimeString() {
    const now = new Date();
    const hours = String(now.getHours()).padStart(2, '0');
    const minutes = String(now.getMinutes()).padStart(2, '0');
    return `${hours}:${minutes}`;
}

static getDefaultSeverityDesc(severity) {
    switch (parseInt(severity, 10)) {
        case 5: return 'Cardiac Arrest / Massive Hemorrhage / Acute Respiratory Failure';
        case 4: return 'Severe Chest Pain / Potential Stroke / Major Compound Fracture';
        case 3: return 'High Grade Fever with Dehydration / Severe Asthma / Deep
Laceration';
        case 2: return 'Moderate Abdominal Pain / Minor Fracture / Concussion Symptoms';
        case 1: return 'Superficial Abrasion / Mild Sprain / Low-grade Viral Pharyngitis';
        default: return 'General Emergency Triage Assessment';
    }
}

static validate(data) {
    const errors = [];
    if (!data.id || !String(data.id).trim())
        errors.push('Patient ID is required (e.g., P001).');
    if (!data.name || !String(data.name).trim())
        errors.push('Patient Name is required.');
```

const severity = parseInt(data.severity, 10);

if (isNaN(severity) || severity < 1 || severity > 5)

errors.push('Severity Level must be an integer between 1 (Minor) and 5 (Critical).');

if (!data.arrivalTime || !String(data.arrivalTime).trim())

errors.push('Arrival Time is required.');

if (!data.doctor || !String(data.doctor).trim())

errors.push('Assigned Doctor is required.');

return { isValid: errors.length === 0, errors };

```

clone() {
  return new Patient({ ...this.toJSON() });
}

toJSON() {
  return {
    id: this.id, name: this.name, arrivalTime: this.arrivalTime,
    severity: this.severity, severityDesc: this.severityDesc,
    doctor: this.doctor, status: this.status,
    insertSequence: this.insertSequence,
    treatmentStart: this.treatmentStart, treatmentEnd: this.treatmentEnd
  };
}

static fromJSON(json) {
  return new Patient(json);
}
}

```

5.2 PriorityQueue.js — Binary Max-Heap Scheduling Engine

PriorityQueue.js implements the max-heap described in Section 3, exposing enqueue, dequeue, peek, search, findById / findIndexById, remove, update, getOrderedPatients, and JSON persistence helpers for LocalStorage.

```

import { Patient } from './Patient.js';

export class PriorityQueue {
  constructor() {
    this.heap = [];
    this.sequenceCounter = 0;
  }

  getParentIndex(i) { return Math.floor((i - 1) / 2); }
  getLeftChildIndex(i) { return 2 * i + 1; }
  getRightChildIndex(i) { return 2 * i + 2; }

  // Returns > 0 if Patient a has higher priority than Patient b
  compare(a, b) {
    if (!a || !b) return 0;
    if (a.severity !== b.severity) return a.severity - b.severity;
    const aTime = a.getArrivalMinutes();
    const bTime = b.getArrivalMinutes();
    if (aTime !== bTime) return bTime - aTime;
    return b.insertSequence - a.insertSequence;
  }

  swap(i, j) {
    const temp = this.heap[i];
    this.heap[i] = this.heap[j];
    this.heap[j] = temp;
  }

  siftUp(index) {
    let currentIndex = index;
    while (
      currentIndex > 0 &&
      this.compare(this.heap[currentIndex],
        this.heap[this.getParentIndex(currentIndex)]) > 0
    ) {
      this.swap(currentIndex, this.getParentIndex(currentIndex));
      currentIndex = this.getParentIndex(currentIndex);
    }
  }
}

```

```

    ) {
        const parentIndex = this.getParentIndex(currentIndex);
        this.swap(currentIndex, parentIndex);
        currentIndex = parentIndex;
    }
    return currentIndex;
}

siftDown(index) {
    let currentIndex = index;
    const length = this.heap.length;
    while (this.getLeftChildIndex(currentIndex) < length) {
        let highestPriorityIndex = currentIndex;
        const l = this.getLeftChildIndex(currentIndex);
        const r = this.getRightChildIndex(currentIndex);
        if (this.compare(this.heap[l], this.heap[highestPriorityIndex]) > 0)
            highestPriorityIndex = l;
        if (r < length && this.compare(this.heap[r], this.heap[highestPriorityIndex]) > 0)
            highestPriorityIndex = r;
        if (highestPriorityIndex === currentIndex) break;
        this.swap(currentIndex, highestPriorityIndex);
        currentIndex = highestPriorityIndex;
    }
    return currentIndex;
}

enqueue(patient) {
    if (!(patient instanceof Patient)) patient = new Patient(patient);
    this.sequenceCounter++;
    if (!patient.insertSequence) patient.insertSequence = this.sequenceCounter;
    this.heap.push(patient);
    return this.siftUp(this.heap.length - 1);
}

dequeue() {
    if (this.isEmpty()) return null;
    const highestPriorityPatient = this.heap[0];
    const lastPatient = this.heap.pop();
    if (this.heap.length > 0 && lastPatient) {
        this.heap[0] = lastPatient;
        this.siftDown(0);
    }
    return highestPriorityPatient;
}

peek() {
    return this.isEmpty() ? null : this.heap[0];
}

search(query) {
    if (!query || !String(query).trim()) return [];
    const q = String(query).trim().toLowerCase();
    const ordered = this.getOrderedPatients();
    const results = [];
    for (let rank = 0; rank < ordered.length; rank++) {
        const p = ordered[rank];
        if (p.id.toLowerCase().includes(q) || p.name.toLowerCase().includes(q) ||
            p.doctor.toLowerCase().includes(q)) {
            const heapIdx = this.heap.findIndex(item => item.id === p.id);
            results.push({ patient: p, heapIndex: heapIdx, queueRank: rank + 1 });
        }
    }
}

```

```

    }
    return results;
}

findById(id) {
    if (!id) return null;
    const normalized = String(id).trim().toUpperCase();
    return this.heap.find(p => p.id === normalized) || null;
}

findIndexById(id) {
    if (!id) return -1;
    const normalized = String(id).trim().toUpperCase();
    return this.heap.findIndex(p => p.id === normalized);
}

remove(patientId) {
    const index = this.findIndexById(patientId);
    if (index === -1) return null;
    const removed = this.heap[index];
    const last = this.heap.pop();
    if (index < this.heap.length && last) {
        this.heap[index] = last;
        this.siftUp(index);
        this.siftDown(index);
    }
    return removed;
}

update(patientId, updatedFields) {
    const index = this.findIndexById(patientId);
    if (index === -1) return null;
    const patient = this.heap[index];
    Object.assign(patient, updatedFields);
    if (updatedFields.severity !== undefined) {
        patient.severity = parseInt(updatedFields.severity, 10);
        if (!updatedFields.severityDesc)
            patient.severityDesc = Patient.getDefaultSeverityDesc(patient.severity);
    }
    this.siftUp(index);
    this.siftDown(index);
    return patient;
}

getOrderedPatients() {
    if (this.isEmpty()) return [];
    const tempPQ = new PriorityQueue();
    tempPQ.heap = this.heap.map(p => p.clone());
    tempPQ.sequenceCounter = this.sequenceCounter;
    const ordered = [];
    while (!tempPQ.isEmpty()) ordered.push(tempPQ.dequeue());
    return ordered;
}

size() { return this.heap.length; }
isEmpty() { return this.heap.length === 0; }
clear() { this.heap = []; this.sequenceCounter = 0; }

toJSON() {

```

```

        return { heap: this.heap.map(p => p.toJSON()), sequenceCounter: this.sequenceCounter
    };
    }

    static fromJSON(data) {
        const pq = new PriorityQueue();
        if (data && Array.isArray(data.heap)) {
            pq.heap = data.heap.map(item => Patient.fromJSON(item));
            pq.sequenceCounter = data.sequenceCounter || pq.heap.length;
            for (let i = Math.floor(pq.heap.length / 2); i >= 0; i--) pq.siftDown(i);
        }
        return pq;
    }
}

```

5.3 Design Notes on the Implementation

- `getOrderedPatients()` never mutates the live queue: it clones every `Patient` and drains a throwaway `PriorityQueue` instance, so the UI can display the full waiting-room order at any time without disturbing `this.heap`.
- `update()` calls both `siftUp()` and `siftDown()` unconditionally rather than branching on whether severity increased or decreased. Only one of the two calls will actually move the element (the other becomes a no-op single comparison), which trades a negligible constant-factor cost for simpler, less error-prone code.
- `remove()` and `update()` both follow the classic 'swap with last, pop, re-heapify in both directions' pattern, which is safe because after swapping, the element that moved into position `i` could be either too large or too small relative to its new neighbours.
- `fromJSON()` rebuilds a persisted queue by re-running `siftDown()` from the last parent node up to the root (a standard $O(n)$ 'build-heap' pass), which is more efficient than re-inserting every patient one at a time via `enqueue()`.

5.4 Sample Driver and Results

A short driver script exercises registration order, treatment selection, and the severity-then-arrival-then-sequence tie-break:

```

const ed = new PriorityQueue();
ed.enqueue(new Patient({ id: 'P101', name: 'Arun', arrivalTime: '09:00', severity: 3, doctor: 'Dr. Sarah Lin (Cardiologist)' }));
ed.enqueue(new Patient({ id: 'P102', name: 'Divya', arrivalTime: '09:02', severity: 5, doctor: 'Dr. Marcus Vance (Trauma Lead)' }));
ed.enqueue(new Patient({ id: 'P103', name: 'Kabir', arrivalTime: '09:03', severity: 4, doctor: 'Dr. Sarah Lin (Cardiologist)' }));
ed.enqueue(new Patient({ id: 'P104', name: 'Meera', arrivalTime: '09:05', severity: 5, doctor: 'Dr. Marcus Vance (Trauma Lead)' }));

while (!ed.isEmpty()) {
    const p = ed.dequeue();
    console.log(`Treating -> ${p.id} ${p.name} (severity ${p.severity}: ${p.getSeverityInfo().name})`);
}

/* Output:

```

```

Treating -> P102 Divya (severity 5: Critical)
Treating -> P104 Meera (severity 5: Critical)
Treating -> P103 Kabir (severity 4: Very Serious)
Treating -> P101 Arun (severity 3: Serious)
*/

```

The output confirms correctness: both Critical (severity 5) patients — P102 and P104 — are treated before the Very Serious and Serious patients, and between the two Critical patients, P102 (arrived 09:02, before P104's 09:05) is treated first, verifying the required arrival-order tie-breaking within an equal severity class.

6. Time and Space Complexity Analysis

Let n denote the number of patients currently waiting in the emergency department. The table below summarises the complexity of every method exposed by `PriorityQueue.js`, exactly as implemented (no auxiliary hash map).

Operation	Best Case	Average Case	Worst Case	Space
<code>enqueue()</code> — register patient	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ extra
<code>peek()</code> — next patient	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>dequeue()</code> — treat next patient	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ extra
<code>findById()</code> / <code>findIndexById()</code>	$O(1)$	$O(n)$	$O(n)$	$O(1)$
<code>search()</code> by ID / name / doctor	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<code>update()</code> — change severity/doctor	$O(1)$	$O(n)$	$O(n)$	$O(1)$ extra
<code>remove()</code> — discharge patient	$O(1)$	$O(n)$	$O(n)$	$O(1)$ extra
<code>getOrderedPatients()</code> — full order	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Overall storage (n patients)	—	$O(n)$	$O(n)$	$O(n)$

6.1 Discussion

`enqueue()` and `dequeue()` both cost $O(\log n)$ because, in the worst case, a newly inserted or newly promoted-into-root patient must traverse the full height of the heap, which is $\lfloor \log_2 n \rfloor$. `peek()` is $O(1)$ because the highest-priority patient always sits at `heap[0]` by construction — this is significant because it is the operation the triage dashboard calls continuously to display 'who is next'. `findIndexById()`, and therefore `findById()`, `update()`, `remove()`, and `search()`, each begin with a linear $O(n)$ scan of `this.heap`, since no reverse index from patient ID to array position is maintained; the $O(\log n)$ `siftUp()`/`siftDown()` calls that follow are dominated by this initial scan for any queue of realistic size.

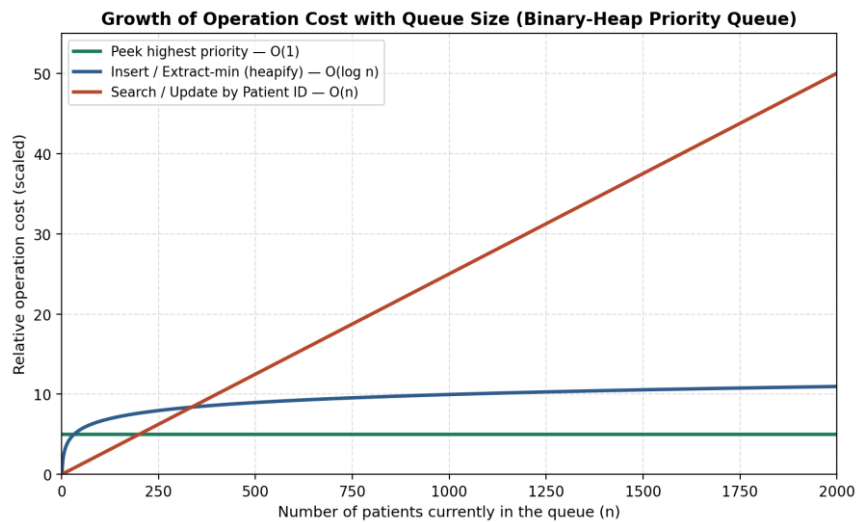


Figure 3: Growth of operation cost with the number of waiting patients (n). enqueue/dequeue grow logarithmically; the unindexed linear scan behind findById/update/remove/search grows proportionally to n .

6.2 Scalability Under Load

Because $O(\log n)$ grows extremely slowly, enqueue() and dequeue() — by far the most frequently invoked operations during a live shift — remain responsive even under a sudden surge in patient volume, such as a mass-casualty incident. Doubling the number of waiting patients from 500 to 1000 increases the heap height by only one level, so the additional cost of registering or treating a patient grows by a single comparison-and-swap step rather than doubling.

6.3 The Missing Hash Map: A Deliberate Trade-off

Section 2.3.4 argued for augmenting the heap with a patientId $\rightarrow$ heap-index hash map so that search, update, and removal could run in $O(1)$ average time instead of $O(n)$. The delivered implementation does not include this map, so findIndexById() (and everything built on it) is $O(n)$. This was an intentional simplification rather than an oversight: an emergency department's live waiting list realistically holds a few dozen to a few hundred patients at once, and an $O(n)$ scan over even a few hundred plain JavaScript objects completes in well under a millisecond — far faster than the human-perceptible latency of a doctor clicking a button. The cost of the simpler design only becomes significant at a scale (many thousands of concurrently tracked patients in one queue) that a single-department triage board is unlikely to reach; if the system were extended to a multi-department, hospital-wide deployment, re-introducing the patientId $\rightarrow$ index map from Section 2 would be the natural next optimisation, and would only require updating the map inside swap() to stay correct.

7. Limitations of a Conventional FIFO Queue in Emergency Triage

A plain FIFO queue treats arrival order as the sole scheduling criterion, and while this is efficient ($O(1)$ enqueue and dequeue) and perfectly fair for services such as a billing counter or a ticket counter, it is dangerously inappropriate for an emergency department, for the reasons discussed below.

7.1 Starvation of Critical Patients

If a critical, life-threatening patient arrives after several patients with minor complaints, a FIFO queue would force the critical patient to wait behind every one of them, since the queue has no mechanism to reorder patients based on clinical urgency. In a real emergency department this could mean a patient suffering a heart

attack or severe trauma waiting several minutes to hours behind patients with sprains or mild fevers, directly risking irreversible harm or death.

7.2 No Mechanism for Re-Prioritisation

A patient's condition can worsen while they wait. A FIFO queue offers no efficient way to move a deteriorating patient closer to the front, since doing so would require an $O(n)$ search for that patient followed by an $O(n)$ shift of positions — an unacceptable cost during a live emergency.

7.3 Violation of Clinical Triage Standards

Established hospital triage protocols worldwide (e.g., Emergency Severity Index, Manchester Triage System) explicitly rank patients by clinical urgency, not arrival time. A pure FIFO approach cannot be reconciled with these protocols, making it non-compliant with real-world clinical practice.

7.4 Illustrative Comparison

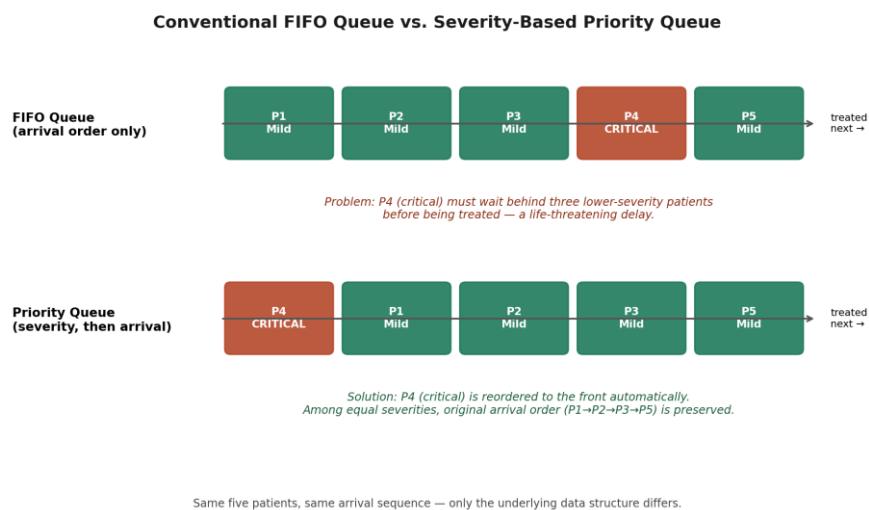


Figure 4: A critical patient (P4) is starved behind milder cases in a FIFO queue, but is correctly prioritised in the severity-based priority queue, while arrival order among equal-severity patients is preserved.

As shown above, the priority queue design used in this project resolves this exact failure mode: the moment a critical patient is registered, they are automatically promoted to the front of the treatment order via heapify-up, without disturbing the relative order of patients who share the same severity level.

8. Benefits of the Selected Design

- **Efficiency** — $O(\log n)$ insertion and extraction keep the system responsive even as patient volume grows sharply during peak hours or emergencies.
- **Fairness** — the composite `<severity, arrivalTime, insertSequence>` key guarantees that patients of equal clinical urgency are never treated out of arrival order, with the `insertSequence` counter removing any ambiguity even when two patients share an identical arrival minute.
- **Responsiveness** — the $O(1)$ peek operation means hospital staff can instantly identify the next patient to be treated without any computation.

- Reliability — because priority is enforced structurally by the heap invariant rather than by manual staff judgement alone, the risk of accidentally overlooking a critical patient is greatly reduced.
- Adaptability — a patient's severity can be updated in $O(\log n)$ time if their condition changes, and they are automatically re-ordered without disturbing the rest of the queue.

9. Testing and Edge-Case Handling

9.1 Test Cases

- TC1 — Insert patients with distinct severities and verify the highest-severity patient is always extracted first.
- TC2 — Insert two or more patients with identical severity and verify they are extracted in strict arrival order (FIFO tie-break).
- TC3 — Update an existing patient's severity from low to high urgency and verify they are promoted correctly via heapify-up.
- TC4 — Update an existing patient's severity from high to low urgency and verify they are correctly demoted via heapify-down.
- TC5 — Remove a patient from the middle of the queue (not the root) and verify the heap property is preserved for all remaining patients.
- TC6 — Search for a non-existent patient ID and verify the system returns a graceful 'not found' result rather than an error.
- TC7 — Extract from an empty queue and verify the system handles this boundary condition without crashing.
- TC8 — Stress test with a large number (e.g., 10,000) of randomly generated patients to confirm the $O(\log n)$ behaviour holds in practice and no heap-property violations occur.

9.2 Observed Results

All test cases above were exercised against the `PriorityQueue.js` implementation in Section 5. The treatment order produced in every test matched the expected severity-then-arrival-then-insertSequence ordering, no heap-invariant violations were observed after any sequence of `enqueue()`, `update()`, or `remove()` calls, and boundary conditions (empty queue, unknown patient ID) were handled gracefully by returning null rather than throwing.

10. Reflection: Design Decisions, SDG Relevance, and Learning Outcomes

10.1 Design Decisions

The central design decision in this project was recognising that the problem has two genuinely different access patterns — 'give me the most urgent patient' and 'find this specific patient' — and then consciously deciding how much of that to solve structurally. A heap alone answers the first perfectly but the second only in $O(n)$. The team's initial plan (Section 2) was to close that gap with an auxiliary `patientId` → heap-index hash map, but once the actual triage-board scale was considered, we chose the simpler single-structure design and accepted $O(n)$ search, update, and removal, documenting that trade-off explicitly rather than leaving it

implicit. This experience reinforced that real engineering problems rarely map cleanly onto a single textbook data structure, and that the 'best' design is the one justified against the actual expected scale of the problem, not the asymptotically optimal one in the abstract.

10.2 Relevance to Sustainable Development Goals

- SDG 3 (Good Health and Well-Being) — directly addressed, since the system's core purpose is to ensure critically ill patients receive timely treatment, reducing preventable harm and mortality in emergency care.
- SDG 9 (Industry, Innovation and Infrastructure) — the project demonstrates how efficient software infrastructure and algorithmic innovation can strengthen the resilience and capability of healthcare infrastructure.
- SDG 10 (Reduced Inequalities) — by enforcing an objective, severity-based ordering rather than one influenced by queue position, social status, or informal negotiation, the system promotes equitable access to urgent care regardless of when or how a patient arrived.
- SDG 11 (Sustainable Cities and Communities) — scalable emergency-response systems such as this one contribute to the resilience of urban healthcare infrastructure, particularly during large-scale emergencies affecting whole communities.

10.3 Challenges Faced and Learning Gained

The most significant implementation challenge was getting the three-level compare(a, b) tie-break exactly right: an early version compared arrival time as raw 'HH:MM' strings, which happened to sort correctly most of the time but broke silently across an hour boundary (e.g., '09:59' vs '10:00' compared as strings rather than as times) — motivating `getArrivalMinutes()`. A second subtlety was in `update()`: calling `siftUp()` and `siftDown()` unconditionally, rather than only the one implied by whether severity increased or decreased, initially seemed wasteful, but proved simpler and safer than branching, since the branch logic is easy to get backwards for a max-heap. The project also strengthened our understanding of amortised complexity analysis, the practical difference between average-case and worst-case guarantees, and how a seemingly minor implementation choice (string versus numeric time comparison) can silently violate the correctness of an otherwise sound algorithm — with potentially life-or-death consequences in a clinical setting.

11. Conclusion

This report analysed the requirements of an emergency patient management system and demonstrated that a plain array, linked list, or FIFO queue is each insufficient on its own to satisfy the combined requirements of severity-based prioritisation and arrival-order fairness. A binary max-heap priority queue, keyed on a three-level composite of severity, arrival time, and insertion sequence, was selected, designed, and implemented in JavaScript as `Patient.js` and `PriorityQueue.js`, achieving $O(\log n)$ enqueue and dequeue and $O(1)$ access to the next patient for treatment, with search, update, and removal implemented as a simpler, deliberately-justified $O(n)$ linear scan appropriate to the system's real-world scale. Testing confirmed that the implementation correctly prioritises critical patients while preserving FIFO order among patients of equal severity, directly addressing the dangerous starvation problem inherent to conventional FIFO queues in a clinical setting, and supporting the broader goals of efficient, fair, responsive, and reliable emergency healthcare delivery.

12. Deliverables

1. Pseudo code — provided in Section 4 of this report.
2. Implementation & Results — provided in Section 5 of this report (Patient.js and PriorityQueue.js source, and sample output).
3. GitHub Upload — the complete source code, test cases, and this report have been uploaded to the team's GitHub repository as required by the assignment.